

NLP 與文字探勘：讓電腦讀懂文字

M6 — Natural Language Processing & Text Mining

護理與健康領域最有價值的資料，多半是文字：病歷、交班紀錄、問卷開放題、衛教諮詢。這一週學怎麼把它們變成可以分析的東西。

本講義大綱

01 為什麼要學這個 · 醫護領域大量高價值資料是文字

02 NLP 的三個時代 · 規則式 → 統計 → 深度學習

03 文字探勘七步流程 · 跟 M1 資料工程的對照

04 前處理逐步實例 · 一句病歷的六個階段

05 ★ 中文斷詞 · 這一步做不好，後面全白做

06 詞袋與 DTM · 文字變成表格

07 TF-IDF · 為什麼「病人」這個詞沒有價值

08 N-gram 與否定偵測 · 把語序救回來

09 關鍵詞抽取 · 找出真正有訊息量的詞

10 文本分類 · Naive Bayes 手算實例

11 主題模型 · 從沒有標籤的文字找主題

12 情感分析 · 三種做法的實際差異

13 命名實體辨識 · 把敘述變成欄位

14 現代方法：零樣本與摘要 · 不用標註資料也能做

15 任務地圖 · 哪個任務該用哪一代方法

16 AI Studio 文字探勘實作 · 課堂範例

17 探索性文字分析 · 拿到資料的第一件事

18 文件相似度與標註品質 · 以及 Kappa 一致性

19 醫護應用案例 · 六個可執行的方向

20 醫療文本的特殊挑戰 · 否定、縮寫、去識別化

為什麼護理與健康科技的學生特別該學文字探勘

你手上最多的資料，其實是文字

- 護理紀錄、交班單、病程摘要、出院準備評估——全是文字
- 問卷的開放題、滿意度調查的意見欄、衛教諮詢的提問紀錄
- ★ 這些資料量大、價值高，但因為「不是數字」而長期被丟在一邊

文字資料的分析價值不輸結構化資料

- 護理紀錄裡的措辭變化，可能比生命徵象更早反映病情惡化
- 滿意度的星等只告訴你「幾分」，開放題才告訴你「為什麼」
- 跌倒與異常事件通報的敘述欄，藏著真正的根因

而且你有一個資工背景的人沒有的優勢

- ★ 你看得懂那些文字。你知道「主訴」「否認」「PRN」是什麼意思
- 主題模型跑出一組詞，資工的人不知道那代表什麼，你知道
- 領域知識在文字探勘裡的權重，比在數值分析裡更高

這一週的目標不是把你變成 NLP 工程師，而是讓你有能力判斷「這批文字能不能分析、該怎麼分析」。

PART 01

NLP 的三個時代

70 Years of NLP

七十年來 NLP 換過三種完全不同的做法。
但新的沒有淘汰舊的——這一點對醫療場域特別重要。

NLP 的三個時代

NLP 的三個時代：七十年的演進



★ 重點：新時代沒有淘汰舊時代。醫療場域今天仍大量使用規則式（如關鍵字警訊偵測）與 TF-IDF —— 因為它們可解釋、可稽核、不用算力。

規則式 (1950s–80s)

人工寫文法規則、關鍵字比對。
完全可解釋，但規則會爆炸。

統計與傳統 ML (1990s–2010s)

TF-IDF、Naive Bayes、主題模型。
★ 這一週的主要內容。

深度學習 (2013–今)

Word2Vec → BERT → GPT。懂上下文，
但不易解釋。
M7 會深入。

★ 新的沒有淘汰舊的

醫療今天仍大量使用規則式（關鍵字警訊偵測）
與 TF-IDF——因為可解釋、可稽核、不用算力。

同一個任務「找出有自殺意念的諮詢紀錄」，三代做法對照

做法	怎麼做	優點	問題
規則式	建關鍵字表：「不想活」「結束一切」「沒有意義」等，命中就標記	★ 完全可稽核；漏掉哪個詞看得出來；部署成本低	換句話說就抓不到；「我不想活在這種痛裡」與「我不想活了」無法區分
傳統 ML	把標註過的紀錄做成 TF-IDF，訓練分類器	能學到關鍵字表沒想到的詞；可算出各詞的權重	需要幾百筆標註資料；仍看不懂語序與否定
現代方法	用預訓練模型做零樣本或少樣本分類	不需要（或只需要極少）標註；理解上下文與委婉表達	需額外說明判斷依據與不確定性；高風險場景必須驗證與覆核

★ 實務常見做法：三者並用。規則式先攔截已知高風險詞，模型補捉上下文與委婉表達，最後由人工覆核；任何一層都不能保證零漏失。

PART 02

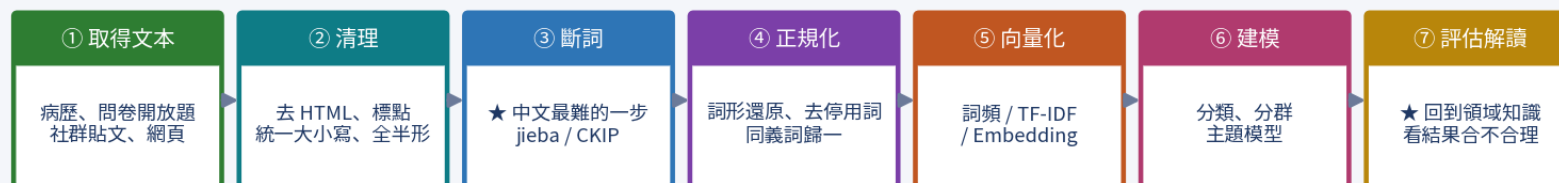
文字探勘的完整流程

The Text Mining Pipeline

七個步驟。實務上大量時間常花在前三步：資料取得、清理與切分；中文尤其需要仔細驗證。

七步流程，以及跟 M1 的對照

文字探勘的完整流程：七個步驟



跟 M1 的資料工程流程對照著看

- ②③④ 就是 M1 的「資料清理」——只是對象從數字欄位換成文字
- ⑤ 向量化 = M1 的「特徵工程」——把非結構化文字變成模型吃得下的矩陣
- ⑥⑦ 之後接的就是 M2/M3 學過的分類器與分群法，完全沿用

★ 八成的時間花在 ②③④。這不是誇飾——中文文本尤其如此。

很多人以為文字探勘的難點在演算法，其實難點在「把字變乾淨」。演算法反而是現成的。

②③④ = M1 的資料清理

只是對象從數字欄位換成文字。

⑤ 向量化 = M1 的特徵工程

把非結構化文字變成模型吃得下的矩陣。
★ 這是整個流程的核心轉換。

⑥⑦ 可沿用 M2 / M3 的評估框架

向量化後可以接分類器與分群法；但表示法、資料切分與評估仍要依文字特性設計。

★ 難點不在演算法

難點在資料品質、表示法與驗證；演算法只是整體流程的一部分。

前處理逐步實例：一句病歷的六個階段

前處理逐步實例：一句病歷的六個階段

原始文本	病人 78 歲，主訴「胸痛」三天，於 2027/3/5 就診。 否認發燒。 從系統匯出的原始字串，含 HTML 標籤與全形符號
① 清理	病人 78 歲 主訴 胸痛 三天 於 2027/3/5 就診 否認發燒 去 HTML、去標點、全形轉半形
② 斷詞	病人 78 歲 主訴 胸痛 三天 於 2027/3/5 就診 否認 發燒 ★ 中文沒有空格，這一步決定後面所有結果
③ 去停用詞	病人 78 歲 主訴 胸痛 三天 就診 否認 發燒 移除「於」等無實質意義的詞
④ 正規化	病人 歲 主訴 胸痛 天 就診 否認 發燒 + [年齡:78] [日期:2027-03-05] 數字與日期抽成結構化欄位，不留在詞袋裡
⑤ 向量化	{病人:1, 主訴:1, 胸痛:1, 就診:1, 否認:1, 發燒:1, ...} ★ 注意：「否認發燒」被拆成兩個詞，語意已經丟失
★ 最後一列是醫療 NLP 的核心風險：詞袋模型丟掉語序後，「否認發燒」與「發燒」在向量上幾乎沒有差別。	

從系統匯出的原始字串

含 HTML 標籤、全形符號、日期、數字——都要處理。

★ 斷詞是分水嶺

中文沒有空格。這一步決定後面所有結果。

數字與日期要抽出來

年齡、日期、檢驗值應該變成結構化欄位，不要留在詞袋裡當成一般的詞。

★ 最後一列是核心風險

詞袋丟掉語序後，「否認發燒」與「發燒」在向量上幾乎沒有差別。

前處理的六個技巧：做什麼、什麼時候不要做

技巧	做什麼	什麼時候不要做
轉小寫 Lowercasing	統一大小寫，避免 Fever 與 fever 被當成兩個詞	★ 需要辨識縮寫時（US 美國 vs us 我們）；藥名大小寫有意義時
去標點 Punctuation Removal	移除標點符號	做情感分析時（「!!!」有意義）； 要保留句界時
斷詞 Tokenization	把文字切成詞	傳統 DTM / TF-IDF 通常需要；字元或 Transformer 子詞模型另當別論
去停用詞 Stop Words	移除「的」「了」「是」等無意義的詞	★ 做否定偵測時（「不」「無」「否認」常在停用詞表裡！）
詞形還原 Lemmatization	running → run，把詞還原成原形	中文沒有詞形變化，這一步主要用於英文
詞性標註 POS Tagging	標出名詞、動詞等，可用來篩選	多數任務不需要；想只看名詞做主題分析時才用

★ 醫療文本常見陷阱：停用詞表可能包含否定詞。先確認任務與處理順序，不要直接套用通用清單。

PART 03

中文斷詞

Chinese Word Segmentation

英文多以空格標示詞界，中文通常沒有。對詞袋、TF-IDF 等詞級方法，斷詞品質會直接影響特徵。

中文斷詞的難題與兩個主要工具

中文斷詞：這一步做不好，後面全白做

英文天生有空格；中文沒有。同一串字可以有多種切法，而切法不同結果就完全不同。

「乾女兒」

乾 | 女兒 vs 乾女兒

一般詞彙的歧義

「研究生命的起源」

研究生 | 命的起源 vs 研究 | 生命 | 的 | 起源

★ 經典例子，切錯語意全變

「病人服藥後不適」

病人 | 服藥 | 後 | 不適 vs 病 | 人服 | 藥後 | 不適

醫療文本的斷詞風險

「二尖瓣脫垂」

二 | 尖 | 瓣 | 脫垂 vs 二尖瓣 | 脫垂

★ 專有名詞未收錄就會被切碎

jieba (結巴)

- 簡體中文語料訓練，繁體與台灣用語較弱
- 輕量、快、易加自訂詞典
- 三種模式：精確／全模式／搜尋引擎
- 適合：社群貼文、非正式文本、需頻繁加詞

CKIP Transformers (中研院)

- ★ 以臺灣繁體語料訓練，本地用語準確
- 同時提供詞性標註與命名實體辨識
- 較重、需下載模型，速度較慢
- 適合：★ 醫療、公文、學術等嚴謹文本

★ 醫療文本的關鍵動作：把專科術語做成自訂詞典餵給斷詞器。不做這件事，「二尖瓣脫垂」會被切成四個沒意義的字。

歧義切分

「研究生命的起源」可以切成「研究生 | 命的起源」，
也可以切成「研究 | 生命 | 的 | 起源」。

jieba (結巴)

簡體語料訓練，台灣用語較弱。輕量、快、易加詞典。
適合社群貼文等非正式文本。

★ CKIP Transformers (中研院)

提供繁體中文斷詞、詞性與命名實體辨識。
適合臺灣文本起步，但醫療術語仍要用自家語料驗證。

★ 最關鍵的動作

詞典式工具可加入專科術語；
Transformer 斷詞器則需微調、後處理或實體詞表輔助。

中文斷詞 ≠ Transformer 子詞切分

方法	輸出單位	主要目的	限制	適用情境
中文斷詞 Word Segmentation	切出可讀詞語	建立 DTM / TF-IDF	詞界標準可能不同	傳統文字探勘
子詞切分 Subword Tokenization	切成字元與子詞	映射模型詞彙 ID	不等同可讀詞語	BERT / LLM
二者並用 Hybrid	保留詞界再做 token	分析詞級結果與對齊	流程較複雜	NER / 可解釋性

★ 斷詞對 DTM / TF-IDF 很重要；Transformer 仍需要配套 tokenizer，但不等於先做 jieba。

實作：中文斷詞與自訂詞典

重點在最後那段——自訂詞典帶來的差異

Python (課後自學；課堂以 AI Studio 為主)

```
# — 方法一：jieba ( 輕量、快 ) —————
import jieba

text = '病人主訴二尖瓣脫垂合併心房顫動，否認胸痛'
print('/'.join(jieba.cut(text)))
# 病人/主訴/二/尖/瓣/脫/垂/合併/心房/顫動/ /否認/胸痛
#           ↑ 專有名詞被切碎

# ★ 加入自訂詞典後
jieba.load_userdict('medical_terms.txt')  # 一行一個詞
print('/'.join(jieba.cut(text)))
# 病人/主訴/二尖瓣脫垂/合併/心房顫動/ /否認/胸痛
#           ↑ 正確了

# — 方法二：CKIP Transformers ( 繁中準確度較佳 ) —————
from ckip_transformers.nlp import CkipWordSegmenter, CkipPosTagger

ws = CkipWordSegmenter(model='bert-base')
pos = CkipPosTagger(model='bert-base')
words = ws([text]); tags = pos(words)
# 同時得到斷詞與詞性，且對臺灣用語與繁體較準

# medical_terms.txt 格式：一行一個詞 ( 可加詞頻與詞性 )
#   二尖瓣脫垂 100 n / 心房顫動 100 n / 慢性阻塞性肺病 100 n
```

建立醫療自訂詞典的四個來源（上）

① 現成的標準詞表

- ICD-10 診斷碼的中文名稱、健保藥品品項檔、LOINC 檢驗項目
- 衛福部與各學會公布的名詞對照表
- ★ 這些多半可公開下載，是最省力的起點

② 從你自己的資料中挖

- 先用預設詞典跑一次，把「被切成單字的連續序列」找出來人工檢視
- 常出現的字串組合往往就是漏掉的專有名詞
- 統計方法：互資訊 (PMI) 與左右鄰字熵可自動偵測新詞

這兩個來源可以讓你在半天內建出一份堪用的詞典。

建立醫療自訂詞典的四個來源（下）

③ 科內的慣用語與縮寫

- ▶ 每個單位都有自己的簡寫：BP、I/O、NPO、on Foley
- ▶ ★ 這些外面的詞典絕對不會有，只能你自己整理
- ▶ 順帶把同義詞歸一：「高血壓 / HTN / hypertension」對到同一個詞

④ 迭代維護

- ▶ 詞典不是做一次就好。新藥、新術式、新縮寫會一直出現
- ▶ 把它當成要持續維護的資產，而不是一次性的前處理
- ▶ 建議：每次分析發現被切碎的詞就補進去，累積成科內資產

★ 對詞袋與 TF-IDF 流程，自訂詞典常是高投報率的改進；仍應用固定測試集驗證是否真的提升。

PART 04

向量化：文字變成數字

Vectorization

整個文字探勘流程的核心轉換就在這裡。
轉完之後，你就回到 M2 / M3 熟悉的世界了。

Document-Term Matrix：文字變成表格

Document-Term Matrix（文件—詞矩陣）：文字變成表格

D1 病人主訴 胸痛 三天
D2 病人主訴 頭痛 兩天
D3 病人否認 胸痛 但有 頭痛
D4 護理 指導 衛教 出院 用藥

	詞 (Terms) →							
	病人	主訴	胸痛	頭痛	天	否認	護理	衛教
D1	1	1	1	0	1	0	0	0
D2	1	1	0	1	1	0	0	0
D3	1	0	1	1	0	1	0	0
D4	0	0	0	0	0	0	1	1

★ 這叫「詞袋」 (Bag of Words) ——因為它真的只是一個袋子，裡面的詞沒有順序。

D1「病人主訴胸痛」與「胸痛主訴病人」在這個矩陣裡完全相同。這是詞袋模型最大的限制。

每一列是一份文件

每一欄是一個詞，格子裡是次數。

這叫「詞袋」 Bag of Words

因為它真的只是一個袋子——裡面的詞沒有順序。

★ 詞袋最大的限制

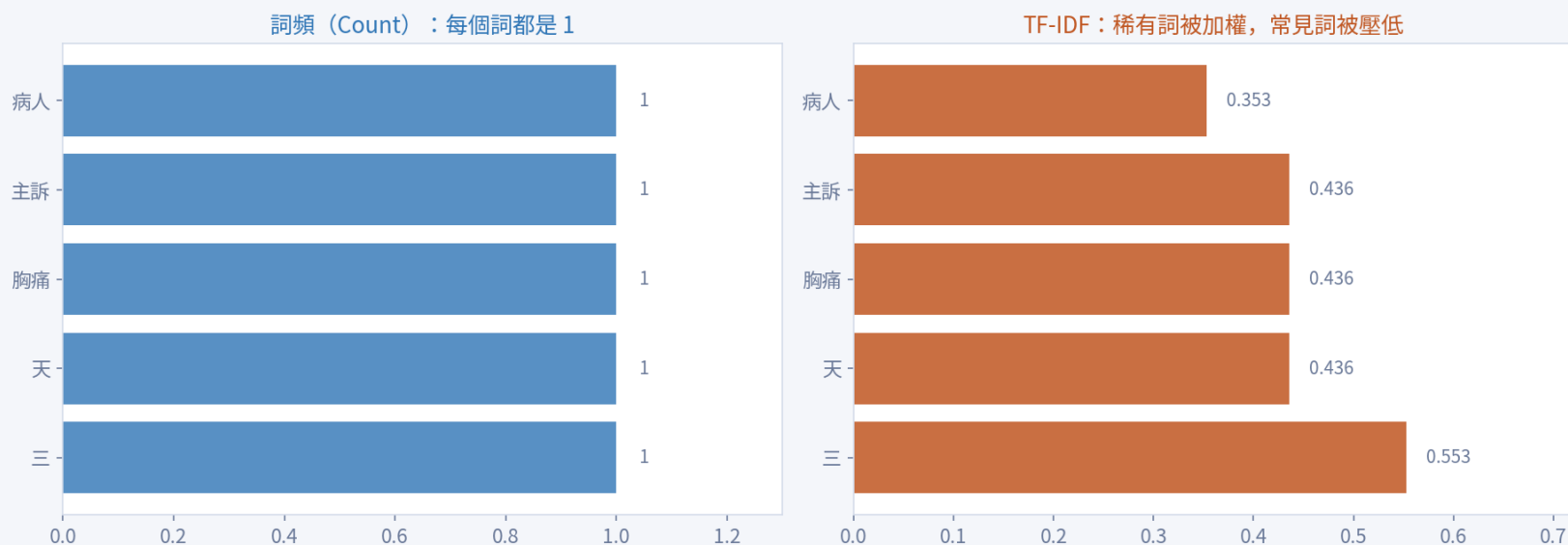
「病人主訴胸痛」與「胸痛主訴病人」在矩陣裡完全相同。
語序資訊全部丟失。

矩陣會很稀疏

真實資料通常是幾千篇 × 幾萬個詞，
而每篇只用到其中幾十個 → 99% 以上是 0。

TF-IDF：為什麼「病人」這個詞沒有價值

同一份文件 D1「病人主訴胸痛三天」：兩種權重的差別



★「病人」出現在 4 篇中的 3 篇 → 沒有鑑別力 → 權重最低 (0.353)。
「三」只出現在 1 篇 → 最能代表這份文件 → 權重最高 (0.553)。這就是 IDF 在做的事。

問題

詞頻把每個詞當成一樣重要。但「病人」出現在每一篇病歷裡，它完全沒有鑑別力。

TF 詞頻

這個詞在這篇文件出現幾次（除以文件長度做正規化）。

IDF 逆文件頻率

出現在越少篇文件中的詞，權重越高。
概念式： $\log(\text{文件總數} / \text{含該詞的文件數})$ ；套件常加入平滑。

★ 結果

本例經 TF-IDF 與 L2 正規化後：「病人」0.353；
「三」0.553。這些不是原始 IDF。

TF-IDF 完整手算：四篇病歷

所有數值皆以 scikit-learn 的公式實際計算並驗證

手算範例 (已用 numpy 驗算)

【語料】四篇 (已斷詞)

D1 病人 主訴 胸痛 三 天
D2 病人 主訴 頭痛 兩 天
D3 病人 否認 胸痛 但 有 頭痛
D4 護理 指導 衛教 出院 用藥

【IDF 計算】 $\text{idf}(t) = \ln((1+N) / (1+\text{df})) + 1$ $N = 4$

詞	df	idf	
病人	3	$\ln(5/4)+1 = 1.223$	← 三篇都有，權重最低
主訴	2	$\ln(5/3)+1 = 1.511$	
胸痛	2	$\ln(5/3)+1 = 1.511$	
頭痛	2	$\ln(5/3)+1 = 1.511$	
天	2	$\ln(5/3)+1 = 1.511$	
三	1	$\ln(5/2)+1 = 1.916$	← 只有一篇有，權重最高

【D1 的向量】 $\text{tf} \times \text{idf}$ ，再做 L2 正規化

三	$1 \times 1.916 = 1.916$	→ 0.553	← 最能代表 D1 的詞
主訴	$1 \times 1.511 = 1.511$	→ 0.436	
胸痛	$1 \times 1.511 = 1.511$	→ 0.436	
天	$1 \times 1.511 = 1.511$	→ 0.436	
病人	$1 \times 1.223 = 1.223$	→ 0.353	← 貢獻最小

★ 注意「三」變成最重要的詞——這其實是個問題 (數字不該當關鍵詞) 。
所以前處理時要先把數字抽成結構化欄位。這就是前面第 ④ 步的用意。

三種向量化方式的比較

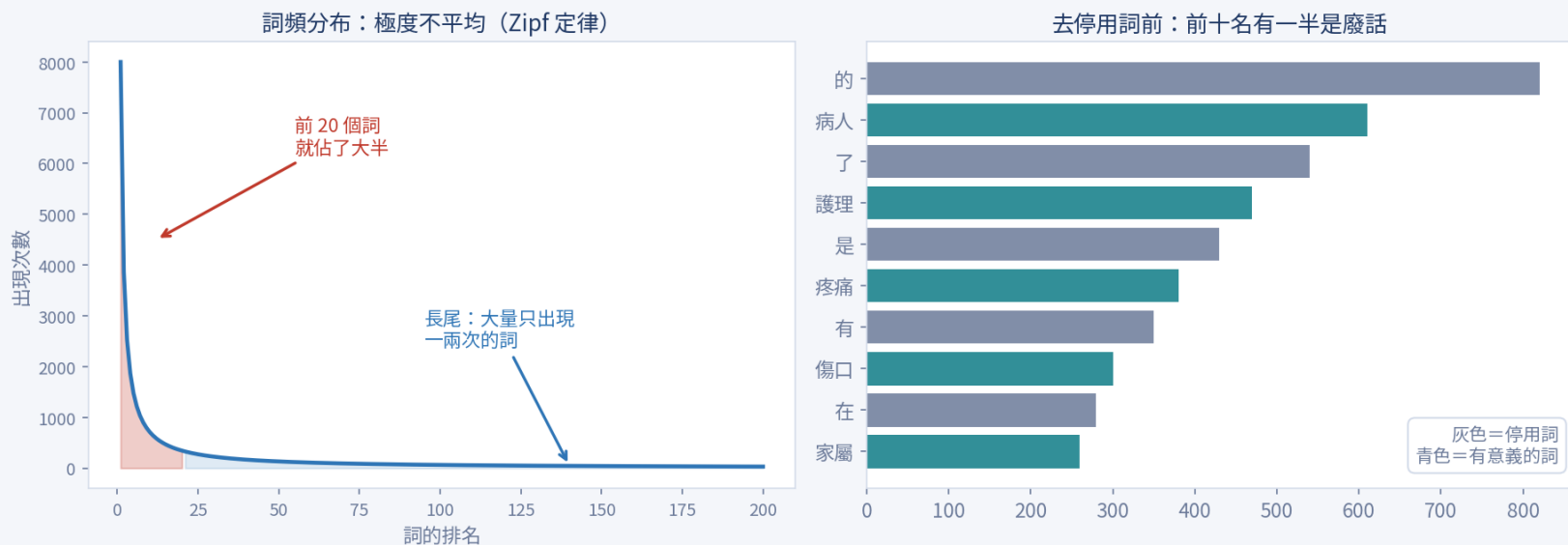
方式	怎麼算	保留什麼	丟掉什麼	什麼時候用
詞頻 Count	數次數	詞的出現與次數	語序、語意、 詞的重要性差異	文件很短、詞彙很受控時
TF-IDF	詞頻 × 逆文件頻率	詞的相對重要性	語序、語意	★ 傳統文字探勘的預設選擇
嵌入 Embedding	用預訓練模型轉成稠密向量	★ 語意 (同義詞會靠近) + 部分上下文	可解釋性	需要語意相似、 或詞彙變化大時 (M7 深入)

★ 一個實用建議：先用 TF-IDF 跑一次當基準線。如果 TF-IDF + 邏輯斯迴歸就夠好，就不需要動用嵌入與深度模型。

探索性文字分析：先看詞頻長什麼樣

這是拿到一批文字之後該做的第一件事

探索性文字分析的第一步：先看詞頻長什麼樣



- ★ 兩個實務啟示：① 不去停用詞，你的分析結果會被「的、了、是」淹沒。
② 長尾的詞出現太少，統計上不可靠——實務上會設「最少出現 3 次」的門檻把它們濾掉。

Zipf 定律

詞頻極度不平均：少數幾個詞佔掉大半，剩下是一條長尾。所有自然語言都是這樣。

★ 啟示一

不去停用詞，你的分析結果會被「的、了、是」淹沒。

★ 啟示二

長尾的詞出現太少，統計上不可靠。
實務上用 `min_df` 過濾極低頻詞；門檻要依語料量與任務驗證。

動手第一步

任何文字探勘專案，先畫這兩張圖。
看不出問題就先別急著建模。

探索階段該看的六個指標

指標	怎麼看	看出什麼
文件數與長度分布	畫長度直方圖	有沒有極短（空白）或極長（複製貼上）的異常紀錄
詞彙量（ vocabulary size ）	去重後的詞數	太小 → 用語高度制式；太大 → 可能斷詞有問題
前 50 高頻詞	長條圖	★ 停用詞夠不夠乾淨；有沒有該加進詞典的術語
只出現 1–2 次的詞佔比	計數	比例過高可能表示錯字或斷詞問題；也可能包含重要罕見術語
共現詞對（ bigram ）	統計相鄰詞對	★ 找出被切碎的專有名詞的最好方法
各類別的詞頻差異	分組比較	哪些詞真的能區分類別——建模前的直覺

★ 第五列特別實用：如果「二尖瓣」與「脫垂」總是相鄰出現，那它們八成該合成一個詞。

N-gram：把語序部分救回來

N-gram：把語序部分救回來

句子：「病人 否認 胸痛」

Unigram (1-gram)

病人 / 否認 / 胸痛

3 個特徵。「否認」與「胸痛」各自獨立，看不出「否認」是在修飾「胸痛」。

Bigram (2-gram)

病人否認 / 否認胸痛

★「否認胸痛」變成一個特徵了——這正是我們要的。

Trigram (3-gram)

病人否認胸痛

整句變成一個特徵。太過細碎——換個主詞就對不上，容易過擬合。

好處

- 部分挽回語序資訊
- ★ 否定詞、程度詞的修飾關係救回來了
- 常見搭配詞 (collocation) 被保留
- 對情感分析與否定偵測提升明顯

代價

- 特徵數暴增：1 萬詞 → 1 億種 bigram
- 矩陣更稀疏，多數 bigram 只出現一次
- 訓練變慢、記憶體吃緊
- ★ 實務：只保留出現 ≥ 5 次的 bigram

★ 實務建議：用 unigram + bigram (sklearn 的 `ngram_range=(1,2)`)，再設最低出現次數門檻。這是投報率最高的設定。

問題回顧

詞袋丟掉語序，「否認胸痛」被拆成兩個無關的詞。

★ Bigram 的價值

「否認胸痛」變成一個特徵——這正是醫療文本需要的。

代價

特徵數暴增、矩陣更稀疏、訓練變慢。

★ 實務設定

先比較 unigram 與 unigram + bigram；
`min_df`、`max_df` 與 `ngram_range` 都應在
驗證集調整。

三種挽救語序的方法，由輕到重

方法	做什麼	成本	效果
N-gram	把相鄰的 2-3 個詞合成一個特徵	低（改一個參數）	★ 能救回否定與常見搭配，多數場景夠用
否定範圍標記	偵測否定詞，把其後 N 個詞加上前綴 (胸痛 → NEG_胸痛)	中（要寫規則）	★ 醫療文本效果顯著；NegEx 是經典做法
語法依存分析	解析句子結構，找出修飾關係	高（要模型與算力）	最精確，但中文的依存分析仍不夠穩定
改用現代模型	Transformer 天生理解語序	高（要 GPU 或 API）	能編碼上下文，但成本與可解釋性需另外評估（M7 深入）

★ 建議：先建立 unigram 基準，再測 bigram、否定標記與上下文模型；用同一資料切分比較效益與成本。

否定偵測：醫療文本最該做的一件事

十行規則，換來分類品質的明顯提升

Python (原理示範；實務可用 negspacy 等套件)

```
# — 概念：NegEx 風格的否定範圍標記
NEG_CUES = ['否認', '無', '未', '沒有', '不曾', '排除', '非']
STOP_MARK = ['.', '。', '但', '然而', '不過'] # 否定範圍的邊界
def mark_negation(tokens, window=5):
    out, neg_left = [], 0
    for t in tokens:
        if t in NEG_CUES:
            neg_left = window # 啟動否定範圍
            out.append(t); continue
        if t in STOP_MARK:
            neg_left = 0 # 遇到邊界就關閉
            out.append(t); continue
        if neg_left > 0:
            out.append('NEG_' + t) # ★ 加前綴，變成不同的特徵
            neg_left -= 1
        else:
            out.append(t)
    return out
# — 效果
mark_negation(['病人', '否認', '胸痛', '.', '主訴', '頭痛'])
# → ['病人', '否認', 'NEG_胸痛', '.', '主訴', '頭痛']
#      ↑ 現在「NEG_胸痛」與「胸痛」是兩個不同的特徵了

# ★ STOP_MARK 很重要：沒有邊界，「否認胸痛，主訴頭痛」會把
#      「頭痛」也標成否定 — 那就完全錯了。
```

否定偵測的三個容易出錯的地方

① 否定詞本身常被停用詞表移除

- ▶ 中文停用詞表幾乎都包含「不」「無」「沒」
- ▶ ★ 先做否定標記，再去停用詞——順序反了就前功盡棄
- ▶ 或者：從停用詞表中把否定詞刪掉

② 否定範圍要設邊界

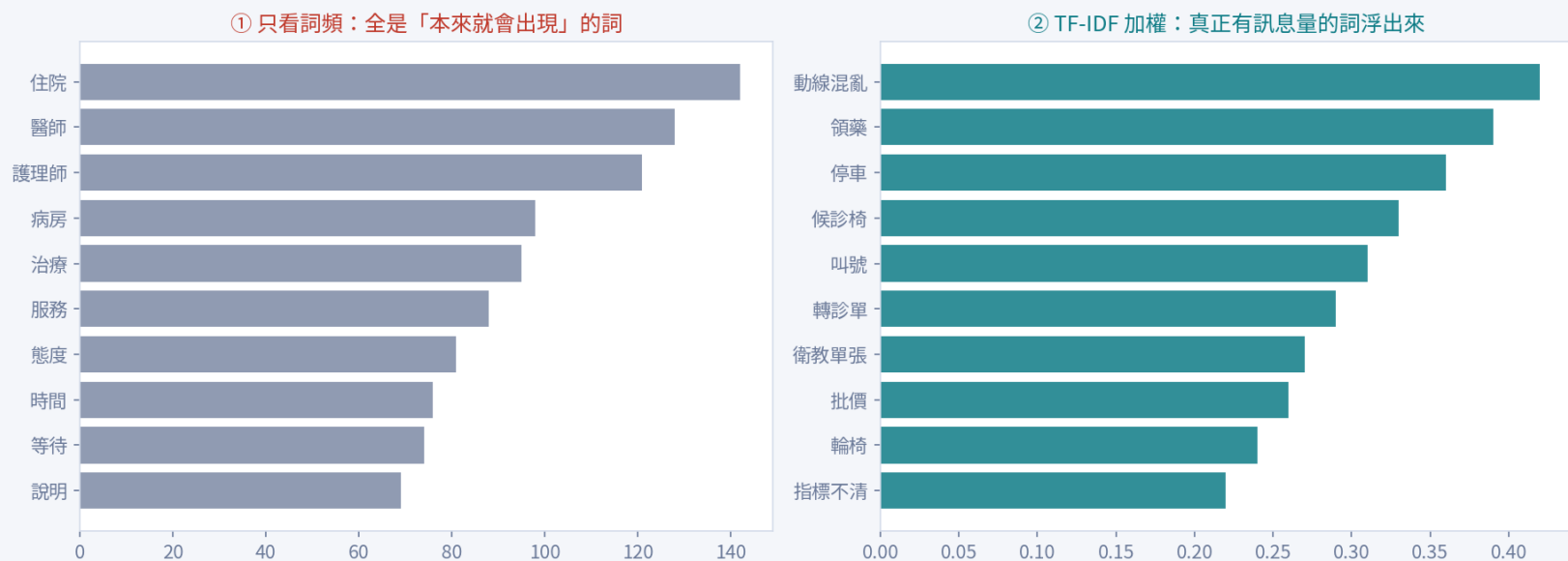
- ▶ 「否認胸痛，主訴頭痛」——「頭痛」不該被標成否定
- ▶ 標點、轉折詞（但、然而）都是天然的邊界
- ▶ 視窗大小（window）建議 3–5 個詞，太長容易誤標

③ 中文的否定表達比想像中多樣

- ▶ 「否認」「無」「未見」「排除」「不曾」「並無」「查無」
- ▶ ★ 還要處理狀態與時間：「已改善」「已緩解」「曾有」「疑似」
- ▶ 建議：從自家語料建立觸發詞、邊界與例外，並用標註集驗證

關鍵詞抽取：找出真正有訊息量的詞

關鍵詞抽取：同一批滿意度留言，兩種排序的天壤之別



★ 左邊那些詞每一則留言都有，看了等於沒看。右邊才是可以拿去開會討論的東西。
做法：對每個類別（或每個時間區間）各算一次 TF-IDF，取前 10 名。

只看詞頻的問題

跑出來的全是「住院、醫師、護理師」——
每一則留言都有，看了等於沒看。

★ TF-IDF 加權之後

「動線混亂、領藥、停車、候診椅、叫號」浮出來了。
這才是可以拿去開會討論的東西。

做法

對每個類別（或每個時間區間）各算一次 TF-IDF，取前 10 名。

延伸

比較「本月」與「上月」的關鍵詞差異，
可以看出問題的變化趨勢。

四種關鍵詞抽取方法

方法	原理	優點	限制
詞頻	數次數	最簡單	適合快速探索，但常被通用高頻詞主導
TF-IDF	相對於整體語料的稀有度	★ 簡單有效，本課程主力	看不出詞與詞的關係
TextRank	把詞當節點、共現當邊，跑 PageRank	考慮詞之間的關聯，單一文件也能用	計算較慢
KeyBERT / 嵌入式	用語意向量找出最能代表全文的詞	理解語意，能抓到換句話說的表達	需要模型與算力 (M7)

★ 一個實用組合：TF-IDF 抓出候選詞 → 人工快速篩一遍 → 得到的清單可以直接當自訂詞典的補充來源。

PART 05

文本分類

Text Classification

向量化之後，M2-2 學過的所有分類器都能直接用。
但有一個演算法特別適合文字——而且它「天真」得可愛。

Naive Bayes 手算實例

Naive Bayes 手算實例：這封信是垃圾信嗎？

訓練資料 (10 封信)

垃圾信 spam : 4 封 | 正常信 ham : 6 封

「免費」出現在 : spam 3 封、ham 1 封

「會議」出現在 : spam 0 封、ham 4 封

待判斷的新信件

「限時免費領取，點擊連結」

含「免費」☑ 不含「會議」☑

計算 (含 Laplace 平滑, 避免機率為 0)

先驗機率	$P(\text{spam}) = 4/10 = 0.4$	$P(\text{ham}) = 6/10 = 0.6$
條件機率	$P(\text{免費} \text{spam}) = (3+1)/(4+2) = 0.667$	$P(\text{免費} \text{ham}) = (1+1)/(6+2) = 0.250$
	$P(\text{會議} \text{spam}) = (0+1)/(4+2) = 0.167$	$P(\text{會議} \text{ham}) = (4+1)/(6+2) = 0.625$
相乘	spam 分數 = $0.4 \times 0.667 \times (1-0.167) = 0.2222$	
	ham 分數 = $0.6 \times 0.250 \times (1-0.625) = 0.0563$	
正規化	$P(\text{spam} \text{這封信}) = 0.2222 / (0.2222 + 0.0563) = 0.798$	

判定：79.8% 是垃圾信 → 分類為 spam

★ 「Naive (天真)」指的是：它假設「免費」與「會議」互相獨立——這在語言上明顯不成立，但它照樣好用。

原理

貝氏定理：看到這些詞，這封信是垃圾信的機率有多高。

Laplace 平滑

「會議」在 spam 中出現 0 次 → 機率為 0
→ 整個乘積歸零。
分子加 1、分母加 2 就避開了這個問題。

結果

$P(\text{spam} | \text{含「免費」不含「會議」}) = 0.798$ → 判為垃圾信。

★ 為什麼叫「天真」

它假設「免費」與「會議」互相獨立。
語言上這明顯不成立——但它照樣好用。

文字分類怎麼選模型：先比較四個強基準

方法	主要優勢	適合情境	注意事項
Multinomial NB Complement NB	訓練快、適合計數與 TF-IDF	小資料與快速基準	需平滑；條件獨立是假設，不是語言事實
Logistic Regression	機率與係數容易說明	需要機率或特徵說明	調整正則化 C、類別權重與決策閾值
Linear SVM / SGD	高維稀疏特徵常有競爭力	中大型語料與高維詞袋	SVM 機率需另校正；仍要調正則化
Zero-shot / LLM	理解上下文與語序	語意複雜或缺標註的起點	成本較高；零樣本仍要本地測試與人工覆核

★ 不用筆數門檻硬選模型。固定切分、向量化與指標，比較 NB / Logistic / Linear SVM，再決定是否升級 Transformer。

完整解讀一次分類結果

完整解讀一次分類結果：數字之外要看什麼

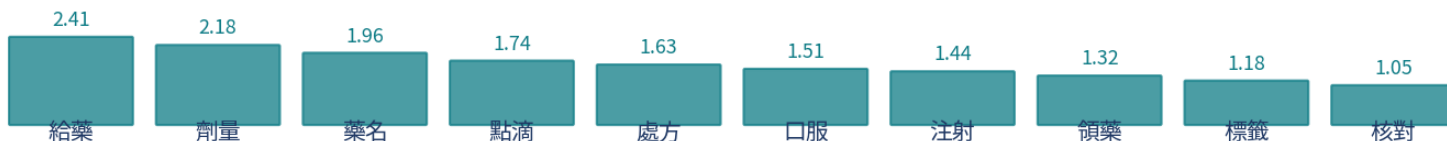
任務：異常事件通報自動分類（是否為「用藥相關」） | 測試集 1,000 筆，其中用藥相關 80 筆

混淆矩陣			指標	
	預測：是	預測：否		
實際：是	62	18	準確率 Accuracy	93.7%
實際：否	45	875	精確率 Precision	57.9%
			召回率 Recall	77.5%
			F1	66.3%

★ 別被準確率騙了

全部猜「否」也有 92% 準確率
（因為 92% 本來就是「否」）
真正該看的是 Recall：
漏掉的 18 筆才是風險所在

模型學到哪些詞？（邏輯斯迴歸係數前十名——這就是可解釋性）



★ 這張係數表是你說服臨床單位的關鍵：它證明模型抓的是對的詞，而不是碰巧猜中。

★ 別被準確率騙了

93.7% 看起來很好，但全部猜「否」也有 92%。
這是 M2-2 教過的類別不平衡陷阱，在文本分類特別常見。

該看 Recall

漏掉的那 18 筆才是風險所在。
醫療場域寧可誤報（45 筆）也不能漏報。

★ 係數表是關鍵

給藥、劑量、藥名、點滴——模型抓的是對的詞。
係數顯示模型依賴哪些關聯詞，但不能證明因果或保證沒有捷徑。

這張表的真正用途

說服臨床單位。「模型準確率 93%」說服不了人，
「模型主要依賴哪些詞，並附上誤判案例」更容易討論。

交付一份文本分類結果時，該附上什麼

最低限度（缺一不可）

- ▶ 混淆矩陣，不是只有準確率
- ▶ ★ Precision / Recall / F1，並說明你優先哪一個、為什麼
- ▶ 類別分布（讓人看得出不平衡程度）
- ▶ 模型看重的前十個詞或特徵
- ▶ 誤判案例：各挑 3–5 筆假陽性與假陰性，附原文

加分項（讓人信任你）

- ▶ 與基準線的比較（全猜多數類、關鍵字規則各是多少）
- ▶ 在不同子群的表現（不同科別、不同時期會不會掉）
- ▶ ★ 標註一致性（Kappa）—— 讓人知道天花板在哪
- ▶ 模型會在什麼情況下失效的誠實說明
- ▶ 後續怎麼監測：上線後準確率掉了誰會發現

★ 最後一項最常被忘記：語言會變、業務會變，今天準的模型半年後可能就不準了。

醫療文本分類的兩個實務提醒

① 類別幾乎總是不平衡

- ▶ 異常事件、跌倒、自殺意念——這些都是少數類別
- ▶ ★ 直接訓練會得到「全部判為正常」的高準確率廢物模型
- ▶ 處理方式沿用 M2-2：調整類別權重、過採樣、改看 Recall
- ▶ 依錯誤代價與覆核量能設定閾值；高風險任務常優先降低漏報

② 要能解釋為什麼

- ▶ 「這份紀錄被判為高風險」——臨床人員一定會問為什麼
- ▶ 邏輯斯迴歸可以列出貢獻最大的前十個詞
- ▶ ★ 深度模型也能做歸因分析，但結果需穩健性檢查（M9 會深入）
- ▶ 實務折衷：模型給分數，同時列出命中的關鍵詞供人判讀

醫療文本怎麼切分資料：先阻斷病人與複製貼上洩漏

① 病人層級切分 Patient-level Split

- ▶ 同一病人的所有紀錄只能在 train 或 test 一側
- ▶ 用 GroupKFold / GroupShuffleSplit 依 patient_id 分組
- ▶ 否則模型可能記住個人或紀錄格式

② 去重與近重複檢查 Deduplication

- ▶ 臨床紀錄常含複製貼上，同一段落可能跨紀錄重現
- ▶ 先找 exact / near duplicates，再決定保留策略
- ▶ 不可讓近重複文本跨 train / test

③ 時間與外部驗證 Temporal / External Validation

- ▶ 接近部署時，以較新時段做鎖定測試集
- ▶ 有條件再做跨科別、跨院或跨系統驗證
- ▶ 報告切分單位、日期範圍與資料來源

PART 06

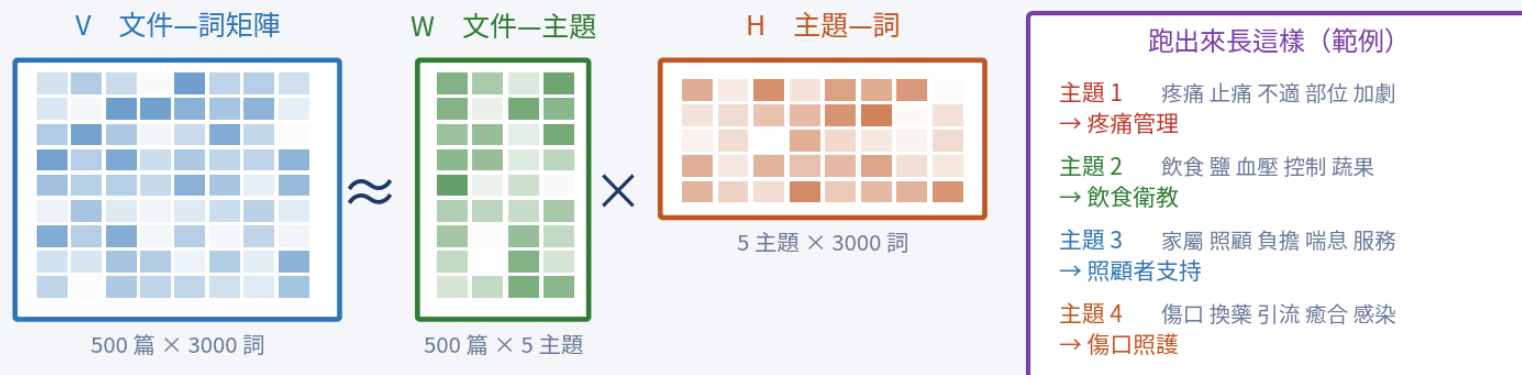
主題模型

Topic Modeling

當你有一堆文字，但完全沒有標籤的時候。
這是文字版的分群——M3 的延伸。

NMF：從沒有標籤的文字中找出主題

主題模型（NMF）：從一堆沒有標籤的文字中找出主題



★ 演算法只給你「詞的組合」，主題的名字是你取的。這一步需要領域知識——這正是護理背景的優勢。

適用：問卷開放題想知道大家在講什麼、病人抱怨分類、衛教需求盤點——所有「有一堆文字但沒有標籤」的情況。

在做什麼

把文件—詞矩陣分解成「文件—主題」與「主題—詞」兩個矩陣。

兩者都做矩陣分解 / 降維，但 NMF 有非負限制，主題解讀方式與 PCA 不同。

輸出是什麼

每個主題就是一組詞。演算法不會告訴你那組詞叫什麼。

★ 命名這一步需要領域知識

看到「疼痛 止痛 不適 部位 加劇」，你知道那是疼痛管理——這正是護理背景的優勢。

適用情境

問卷開放題、病人抱怨分類、衛教需求盤點——
所有「有一堆文字但沒有標籤」的情況。

主題模型怎麼選：看表示法與文件特性

演算法	輸入與特性	特點	注意
NMF 非負矩陣分解	TF-IDF ; 非負矩陣分解	固定初始化與 random_state 時較容易重現	主題數需選擇，並檢查穩定性與可解讀性
LDA 潛在狄利克雷分配	詞頻矩陣； 機率式模型	一篇文件可有主題分布，適合較長文本	結果受 seed、先驗與前處理影響；通常用計數
BERTopic 嵌入式主題模型	嵌入 + UMAP + HDBSCAN + c-TF-IDF	可聚合語意相近短文；需選合適的多語 / 中文嵌入	算力與超參數成本較高；離群文件需檢查

★ 沒有通用的篇數或主題數門檻。比較主題穩定性、coherence、代表文件與領域可解讀性，再決定模型。

主題模型的三個常見誤用

① 共現訊號不足仍硬跑

- ▶ 沒有通用最低篇數；要看文件長度、詞彙與主題分離度
- ▶ 小型語料先做人工主題分析，或與模型結果交叉檢查
- ▶ 用重抽樣或不同 seed 檢查主題是否穩定

② 沒做好前處理就跑

- ▶ 停用詞沒去乾淨 → 每個主題都是「的、了、是」
- ▶ 專有名詞被切碎 → 主題裡全是沒意義的單字
- ▶ ★ 前處理、表示法、模型與參數都會共同影響結果

③ 把主題當成客觀事實

- ▶ 主題是演算法找出的統計模式，不是「真的存在的分類」
- ▶ 換個主題數、換個演算法，結果就會變
- ▶ ★ 正確用法：當成「探索的起點」，幫你發現沒想到的面向，而不是當成結論直接寫進報告

PART 07

情感分析與實體辨識

Sentiment Analysis & NER

兩個在醫護場域最實用的任務：

一個告訴你「病人怎麼想」，一個把敘述變成欄位。

情感分析：三種做法在同一句話上的差異

情感分析：三種做法在同一句話上的差異

測試句：「這次住院的護理人員很細心，不過等待時間實在太久了。」

① 字典法

查情感詞典加總

- 細心 +1 久 -1 太 -1
- 總分 = -1 → 判為「負面」

❌ 錯了。這句話對護理是正面的，只有等待時間是負面。字典法看不出對象。

② 傳統 ML

TF-IDF + 邏輯斯迴歸

- 需要幾百筆已標註的評論訓練
- 輸出：負面 0.62 / 正面 0.38

△ 勉強。看得出整體偏負，但同樣分不出「對什麼」正負。

③ 面向情感分析

現代模型 + 指定面向

- 面向「護理人員」→ 正面 0.94
- 面向「等待時間」→ 負面 0.91

★ 正確。這才是你真正需要的答案：知道哪個環節好、哪個環節該改。

★ 病人滿意度調查的開放題，用整句情感分析幾乎沒有價值——因為一句話裡通常同時誇跟罵。要做面向情感分析。

字典法

查情感詞加總。這句話算出 -1 → 判為負面。

❌ 錯了：對護理是正面的，只有等待時間負面。

傳統 ML

TF-IDF + 分類器。輸出整體偏負。

△ 勉強：同樣分不出「對什麼」正負。

★ 面向情感分析

「護理人員」正面 0.94、「等待時間」負面 0.91。

這才是你真正需要的答案。

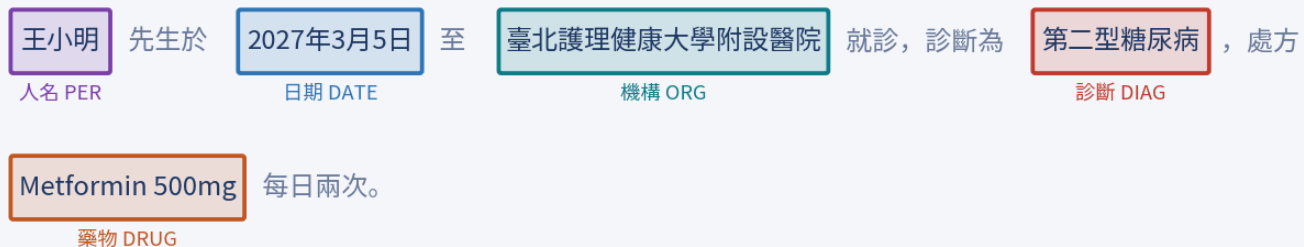
★ 對滿意度調查的意義

一句話可能同時包含正負面。整句分數可能掩蓋細節；需要時再做面向情感分析。

命名實體辨識（NER）：把敘述變成欄位

命名實體辨識（NER）：把敘述變成欄位

輸入：一段門診紀錄



通用 NER 模型認得的

- 人名、地名、機構名
- 日期、時間、數量
- 這些是預訓練模型的標準標籤

醫療 NER 要另外處理的

- ★ 診斷、藥物、劑量、檢驗項目
- 通用模型不認得，要用醫療專用模型
- 或用少量標註資料微調

★ NER 是「病歷結構化」的核心技術：把非結構化的敘述變成可以統計、可以查詢的欄位。
也是「去識別化」的核心技術——先把人名、地址、病歷號找出來，才能遮蔽。這是你做專題前必須先做的一步。

通用模型認得的

人名、地名、機構、日期、數量——預訓練模型的標準標籤。

★ 醫療要另外處理

通用模型未必可靠涵蓋診斷、藥物、劑量與檢驗等臨床實體；需定義標籤架構並做本地驗證。

用途一：病歷結構化

把非結構化敘述變成可統計、可查詢的欄位。

★ 用途二：去識別化

NER 可協助找人名與地址，但還要搭配規則、人工抽查與殘餘風險評估；不能把一次模型輸出當保證。

現代方法：少標註就能起步，但不是零驗證

Hugging Face pipeline：示意程式，模型與輸出需本地驗證

Python (課後自學；概念在 M7 深入)

```
from transformers import pipeline

# ① 零樣本分類 Zero-shot：不需要任何標註資料
clf = pipeline('zero-shot-classification',
               model='joeddav/xlm-roberta-large-xnli')
clf('病人反映等待領藥時間過長，希望能改善動線',
    candidate_labels=['醫療品質', '行政流程', '環境設施', '態度服務'])
# → 示意：行政流程排名最高
# ★ 分數不是校準機率；仍要用代表性標註集測試

# ② 摘要 Summarization：把長文變短
summ = pipeline('summarization', model='YOUR_VALIDATED_MODEL')
summ(long_nursing_note, min_length=40, max_length=120)
# → 用於交班摘要、病程整理
# ★ 摘要可能遺漏、扭曲或加入原文沒有的內容 → 必須人工覆核

# ③ 命名實體辨識 NER
ner = pipeline('ner', aggregation_strategy='simple')
ner('王小明先生於 2027 年 3 月 5 日至北護附設醫院就診')
# → [{'entity_group': 'PER', 'word': '王小明', 'score': 0.998}, ...]
# ★ 去識別化的第一步就是這個

# — 中文要指定中文模型，預設模型多為英文 —
clf = pipeline('zero-shot-classification',
               model='joeddav/xlm-roberta-large-xnli') # 多語言
ner = pipeline('ner', model='ckiplab/bert-base-chinese-ner') # 中研院
```

零樣本、摘要與生成式 NLP：醫療場域的三道閘門

① 任務驗證 Task Validation

- 零樣本不代表零驗證：仍要建立代表性測試集
- 依任務選 Precision / Recall / F1 或人工評分準則
- 先定義可接受錯誤與 go / no-go 門檻

② 隱私與資安 Privacy & Security

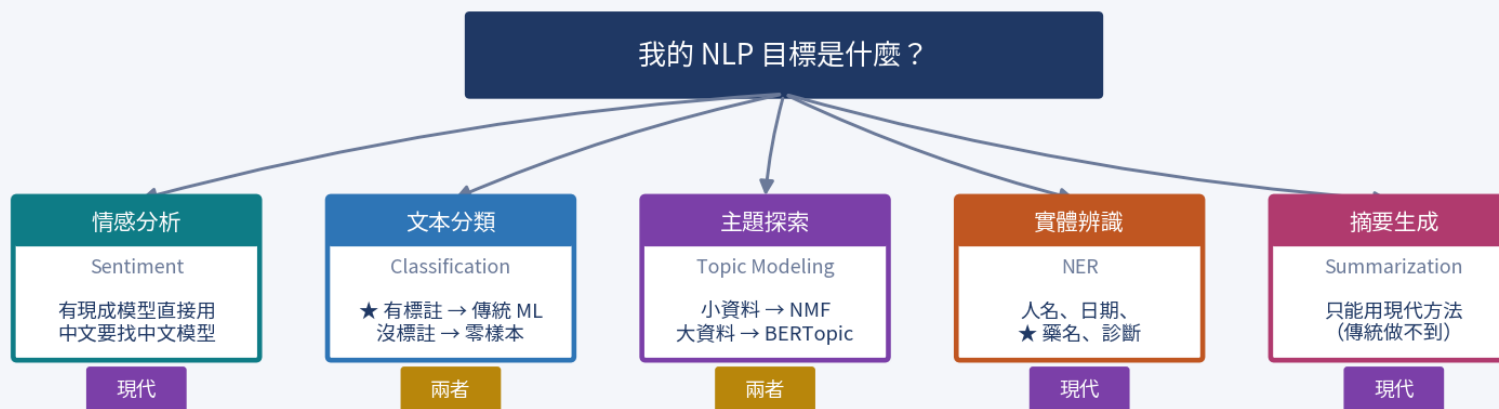
- 不把可識別病人資料輸入未核准的外部服務
- 盤點資料流向、保存政策、存取權限與模型版本
- 防範提示詞注入、資料外洩與供應商變更

③ 人為覆核與監測 Human Oversight

- 生成內容可能遺漏、扭曲或加入原文沒有的資訊
- 臨床用途由具資格的醫事人員最終確認
- 上線後監測錯誤、偏誤、版本漂移與事件

任務地圖：哪個任務該用哪一代方法

NLP 任務地圖：五個常見任務，各自該用哪一代方法



什麼時候用傳統方法 (TF-IDF + ML)

- 資料量小 (< 1 萬筆) 且有標註
- ★ 需要可解釋：能指出是哪些詞造成判斷
- 沒有 GPU、要跑在一般電腦上

什麼時候用現代方法 (Transformer)

- 沒有標註資料 (零樣本 / 少樣本)
- ★ 語序與上下文重要 (如否定、條件句)
- 任務本身需要生成 (摘要、改寫)

★ 不要預設「新的一定比較好」。醫療場域要能解釋為什麼判成這一類時，TF-IDF + 邏輯斯迴歸反而是對的選擇。

有標註 → 傳統方法夠用

資料量小、需要可解釋、沒有 GPU 時，TF-IDF + 邏輯斯迴歸往往是對的選擇。

缺少任務標註 → 可先做零樣本基準

零樣本分類可用未見過的類別名稱起步；仍要用少量代表性資料驗證、調整標籤文字與閾值。

語序重要 → 現代方法

否定句、條件句、委婉表達——詞袋處理不了。

★ 不要預設新的比較好

醫療場域要能解釋判斷依據時，傳統方法反而是正解。

文件相似度與分群

文件相似度與分群：不用標籤就能整理一堆紀錄

五筆護理紀錄

N1 傷口滲液增加，予以換藥

N2 傷口乾淨無滲液，換藥後覆蓋

N3 主訴傷口疼痛，給予止痛藥

N4 家屬詢問出院後照顧方式

N5 已完成出院衛教，家屬了解

分群結果（自動跑出來的）

- 群 1：N1、N2 → 傷口照護／換藥
- 群 2：N4、N5 → 出院準備／家屬衛教
- N3 自成一派 → 疼痛管理（與傷口相關但不同主題）

餘弦相似度矩陣

	N1	N2	N3	N4	N5
N1	1.00	0.82	0.41	0.05	0.08
N2	0.82	1.00	0.35	0.04	0.09
N3	0.41	0.35	1.00	0.06	0.11
N4	0.05	0.04	0.06	1.00	0.71
N5	0.08	0.09	0.11	0.71	1.00

這個技術可以拿來做什麼

- 找重複：判斷兩份交班紀錄是不是在講同一件事（品管稽核常用）
- 自動歸類：把幾千筆異常事件通報依內容自動分群，再人工檢視每一群
- 找相似個案：輸入一段病程，找出過去照護紀錄中最相近的幾筆
- ★ 這就是 M8「知識檢索」的原型——只是那裡用嵌入向量，這裡用 TF-IDF

怎麼算

把每份文件轉成 TF-IDF 向量，兩兩算餘弦相似度。

看得出什麼

N1、N2 相似度 0.82（都在講傷口換藥）；
N4、N5 相似度 0.71（都在講出院衛教）。

實務用途

找重複的紀錄、把幾千筆通報自動分群、找相似個案。

★ 這就是 M8 知識檢索的原型

只是那裡用嵌入向量、這裡用 TF-IDF。概念完全一樣：把問題與文件都變成向量，找最近的。

標註品質：你的訓練資料本身可靠嗎

標註品質：你的訓練資料本身可靠嗎？

情境：兩位護理師各自標註 100 筆紀錄「是否為高風險」

	B：是	B：否	合計
A：是	35	10	45
A：否	5	50	55
合計	40	60	100

綠色=兩人一致（85 筆）

紅色=兩人不一致（15 筆）

Cohen's Kappa 計算

觀察一致率 $P_o = 85/100 = 0.85$

期望一致率 $P_e = (45 \times 40 + 55 \times 60)/100^2$
 $= (1800 + 3300)/10000 = 0.51$

$\kappa = (P_o - P_e)/(1 - P_e)$
 $= (0.85 - 0.51)/(1 - 0.51) = 0.694$

< 0.20

極差

0.21-0.40

尚可

0.41-0.60

中等

0.61-0.80

良好

0.81-1.00

極佳

★ 為什麼不能只看「一致率 85%」：如果兩人都亂猜，光靠機率也會有 51% 一致。Kappa 扣掉了這部分。

為什麼要看這個

標註品質決定模型學到的目標與評估可信度。

★ 標註不一致時，模型可能只是在學習含噪音的共識。

Cohen's Kappa

扣掉「純靠運氣也會一致」的部分之後的真實一致率。

本例 $\kappa = 0.694$ ；需連同類別盛行率、標籤數與任務風險解讀。

★ 為什麼不能只看一致率

兩人都亂猜也會有 51% 一致。

85% 聽起來很高，扣掉之後其實只有 0.694。

κ 太低怎麼辦

不是換模型，是回頭把「標註規則」寫清楚，再重新標一次。這一步省不掉。

做標註的實務建議（上）：開始之前

① 先寫標註手冊，再開始標

- ▶ 每個類別要有定義、正例、反例，以及「模稜兩可時怎麼辦」
- ▶ ★ 最容易吵架的永遠是邊界案例——手冊要先把邊界講清楚
- ▶ 手冊本身就是研究方法的一部分，論文裡要交代

② 兩人獨立標一小批，先算 Kappa

- ▶ 先雙人獨立標一批，計算一致性並看混淆類別
- ▶ 門檻要依任務、類別與風險預先訂定，0.6 不是通則
- ▶ 討論不一致案例、修訂手冊，再進入正式標註

這兩步是整個監督式文字分析中最容易被跳過、也最不該跳過的部分。

做標註的實務建議（下）：過程與收尾

③ 不一致的案例怎麼處理

- ▶ 找第三人仲裁，或兩人討論到達成共識
- ▶ 把仲裁結果回饋進手冊，避免同類型的爭議再吵一次
- ▶ 保留爭議案例清單——它們往往是最有研究價值的部分

④ 記錄標註者資訊與時程

- ▶ 誰標的、什麼時候標的、一次標了多久
- ▶ ★ 疲勞可能影響品質：記錄時程、抽查批次差異並安排休息
- ▶ 建議每次以 50 筆為一批，中間休息

※ 這一節的內容在護理研究的質性資料編碼上完全通用，不只用於機器學習。

PART 08

AI Studio 文字探勘實作

Hands-on with AI Studio

把前面所有概念在 AI Studio 裡實際跑一次。
本模組的範例是課程既有教材中最完整的一組。

課堂實作 | Amazon Alexa 評論的情感分析

★ 對應 AI Studio Repository / 7. Text Mining

開啟檔案

7. Text Mining/

1 Creating Text objects.rmp

1.2 amazon_alex.csv

7 Processing Text and Document Objects.rmp

Case Study 1 Amazon_alex.csv

sentiment analysis.rmp

Case Study 3 Text mining -

Lemmatizer TextClassification

Sentiment .rmp

預期結果

- ▶ 前處理每開一個運算子，詞表會明顯變小
- ▶ TF-IDF 通常略優於單純詞頻
- ▶ 類別若不平衡，準確率會虛高
- ▶ Lemmatizer 對英文有效，對中文無效

操作步驟

- 1 開啟「1 Creating Text objects」，看 Text 物件怎麼建立
- 2 執行「7 Processing Text and Document Objects」
- 3 ★ 逐一開關 Tokenize / Filter Stopwords / Stem 運算子，觀察詞表變化
- 4 執行 Case Study 1，看整體流程：讀檔 → 前處理 → 向量化 → 分類
- 5 把 Process Documents 的向量化方式從 TF-IDF 改成 Term Occurrences，比較準確率
- 6 ★ 檢查混淆矩陣：正負評論的數量是否平衡？Recall 各是多少？
- 7 Case Study 3 加了 Lemmatizer，比較它帶來多少提升

本模組的 AI Studio 範例對照

範例檔	示範什麼	對應本講義
1 Creating Text objects.rmp	建立 Text 物件、讀入文字資料	PART 2
7 Processing Text and Document Objects.rmp	★ 前處理運算子逐一示範	PART 2、PART 3
3. Web Mining from URLs and Tokenize.rmp	從網頁抓文字並斷詞	PART 2
4 / 5 / 6 Automatic Classification of Documents	文件分類與套用模型 (含向量化版本)	PART 5
2 Find Text Association Rules.rmp	詞與詞的關聯規則	延伸
3 Document Similarity and Clustering.rmp	文件相似度與分群	PART 4、PART 6
Case Study 1 Amazon_alexia sentiment analysis.rmp	★ 完整情感分析流程	PART 5、PART 7
Case Study 2 IMDB with Process Documents	較大資料集的文本分類	PART 5
Spam Email Classification.rmp (4. Case Studies)	★ Naive Bayes 垃圾信分類，對應手算範例	PART 5

PART 09

醫療文本的四個特殊挑戰

Clinical Text: What Makes It Hard

通用 NLP 工具在醫療文本上會踩到的坑。

這一節是本模組最該記住的部分。

四個挑戰與對應做法

挑戰	實例	為什麼難	怎麼處理
否定與不確定	「否認胸痛」「疑似肺炎」 「無發燒」	詞袋模型丟掉語序， 「否認胸痛」與「胸痛」幾乎相同	★ 用否定偵測規則（如 NegEx） 或改用理解上下文的現代模型
縮寫與同義詞	BP / 血壓 / Blood Pressure HTN / 高血壓	同一個概念多種寫法， 被當成不同的詞	建同義詞對照表，前處理時歸一
專有名詞被切碎	「二尖瓣脫垂」→ 二 / 尖 / 瓣 / 脫垂	一般詞典沒有收錄醫療術語	★ 自訂詞典（本模組 PART 3）
個資與去識別化	姓名、病歷號、地址、 聯絡電話	一旦外流無法收回； 法規與倫理要求	NER + 規則 + 人工抽查； 再評估殘餘識別風險

★ 否定、時間、不確定性與主體都屬臨床脈絡；只抓詞面可能同時造成誤報與漏報。

去識別化：先分清法律框架、技術處理與殘餘風險

美國 HIPAA Safe Harbor 的 18 類識別資訊

- 包含姓名、地址、電話、病歷號、網址、IP、照片等
- 日期只保留年；超過 89 歲合併為「90 歲以上」
- ★ 這是美國規則，不能直接當成臺灣通則

常見技術處理

- 遮蔽 (Redaction)：刪除或改成 [姓名]、[日期]
- 代換 (Surrogate / Pseudonymization)：保留格式或關聯
- 若仍有對照鍵，就不是不可逆匿名化

★ 臺灣專案仍要做合規與風險評估

- 個資法涵蓋可直接或間接識別的病歷與醫療資料
- 依 IRB、資料使用契約、院內政策與安全環境辦理
- ★ 課程一律使用公開或虛構資料，不碰真實病歷

PART 10

醫護場域的應用案例

Applications in Healthcare

把前面所有技術組合起來，實際上能做什麼。

以下六個都是可以當成期末專題或碩論題目的方向。

六個可執行的應用方向

應用	用到的技術	資料來源	難度
病人滿意度開放題分析	★ 面向情感分析 + 主題模型	院內滿意度調查（需去識別化）或公開評論資料	中
異常事件通報自動分類	斷詞 + TF-IDF + 分類器 （或零樣本分類）	通報系統的敘述欄；可先用虛構資料建模	中
衛教需求盤點	主題模型 + 詞頻分析	★ 衛教諮詢紀錄、線上問答平台的公開提問	低
護理紀錄品質稽核	文件相似度（找複製貼上） + 規則式關鍵欄位檢查	護理紀錄（需去識別化）	中
文獻回顧輔助	關鍵詞抽取 + 文件分群 + 摘要	★ PubMed 等公開資料庫，完全無個資疑慮	低
長照個案需求分析	NER（抽取需求項目）+ 分群	★ 長照相關公開問答與法規文件	中

★ 標星號的三項資料完全公開，沒有個資問題，最適合當作期末專題或第一個練手的題目。

一個完整案例的想像：滿意度開放題分析

如果只看星等

- ▶ 「本月平均 4.2 分，比上月下降 0.3 分」
- ▶ 然後呢？你不知道是哪裡出了問題
- ▶ 主管會問「為什麼降？」，你答不出來
- ▶ ★ 星等只告訴你「幾分」，不告訴你「為什麼」

加上文字分析之後

- ▶ 主題模型跑出五個主題：等待時間、護理態度、環境、說明清楚度、費用
- ▶ 面向情感分析：護理態度 +0.91、等待時間 -0.87
- ▶ 時間趨勢：等待時間的負面比例從 12% 升到 31%
- ▶ ★ 結論具體到可以行動：問題在動線與排程，不在人員態度

這就是文字探勘的價值：把「一個分數」變成「一份可以據以改善的診斷」。

開始一個文字探勘專題前，先問自己五個問題（上）

① 資料拿得到嗎？合法嗎？

- ▶ ★ 這是第一個問題。拿不到或不能用，後面全是空談
- ▶ IRB、資料使用同意、去識別化——這些通常比分析更花時間
- ▶ 課程建議：先用公開資料把方法跑通，再談院內資料

② 資料量與標註品質足夠嗎？

- ▶ 沒有通用最低筆數；看類別數、文本多樣性與錯誤代價
- ▶ 畫 learning curve，確認增加資料是否仍帶來提升
- ▶ 小型資料可先人工分析，或把模型當探索工具而非結論

這兩題決定「這個題目做不做得成」。過不了就換題目，不要硬做。

開始一個文字探勘專題前，先問自己五個問題（下）

③ 我要的答案，真的需要文字分析嗎？

- ▶ 有時候一個結構化欄位就能回答，不必大動干戈
- ▶ ★ 先問「這個答案會改變什麼決策」，再決定要做多複雜

④ 結果要給誰看？他看得懂嗎？

- ▶ 給臨床單位看的東西要能解釋為什麼——這會影響你選哪一代方法
- ▶ ★ 一個準確率 92% 但說不出理由的模型，在臨床往往推不動

⑤ 錯了會怎樣？

- ▶ 高風險場景（自殺意念、異常事件）要留人工覆核，不能全自動
- ▶ 設計時就要想好：模型判錯的那幾筆，誰會發現、怎麼補救

★ 這五題不只適用於文字探勘。任何資料科學專案開始前都值得問一遍。

從 TF-IDF 到詞向量：M6 與 M7 的分界線

從 TF-IDF 到詞向量：M6 與 M7 的分界線

TF-IDF 的世界 (M6)

「醫師」與「醫生」是兩個完全無關的欄位。



彼此之間沒有「距離」的概念

詞向量的世界 (M7 預告)

語意相近的詞，向量位置也相近



詞向量最著名的性質：可以做「運算」

$$\text{vec}(\text{國王}) - \text{vec}(\text{男}) + \text{vec}(\text{女})$$

$$\approx \text{vec}(\text{女王})$$

$$\text{vec}(\text{醫師}) - \text{vec}(\text{醫院}) + \text{vec}(\text{學校})$$

$$\approx \text{vec}(\text{教師})$$

$$\text{vec}(\text{阿斯匹靈}) - \text{vec}(\text{止痛})$$

$$\approx \text{vec}(\text{抗凝血}) \text{ 方向}$$

這代表向量真的
捕捉到了語意結構，
不只是統計共現。

★ Word2Vec (2013) 是這條路的起點；M7 的 Embedding 與 Transformer 就是它的後代。

TF-IDF 的世界

「醫師」與「醫生」是兩個完全無關的欄位。
彼此之間沒有「距離」的概念。

詞向量的世界

語意相近的詞，向量位置也相近。
「醫師」與「醫生」會靠在一起。

★ 常見教學示例 (不是保證)

$$\text{vec}(\text{國王}) - \text{vec}(\text{男}) + \text{vec}(\text{女}) \approx \text{vec}(\text{女王})$$

類比是否成立取決於語料與訓練，也可能反映資料偏誤。

Word2Vec (2013) 是起點

它建立「用向量表示語意」的主線；
Transformer 的上下文嵌入是後續重要發展。

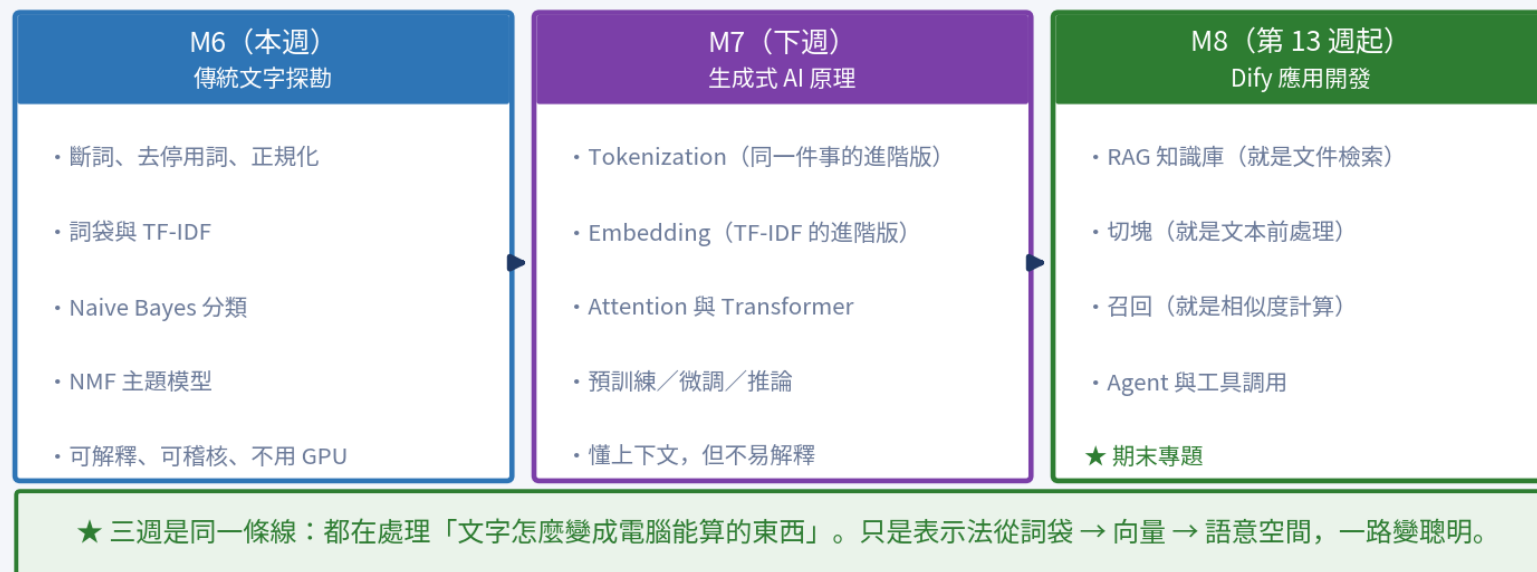
三種文本表示法的完整對照（本模組總結）

	詞頻 / TF-IDF (M6)	詞向量 Word2Vec	上下文向量 BERT (M7)
維度	幾千 ~ 幾萬 (稀疏)	常見 100–300 (稠密)	依模型而定 (如 BERT-base 768)
同義詞	★ 視為完全無關	位置相近	位置相近
一詞多義	無法區分	★ 無法區分 (一詞一向量)	★ 可區分 (依上下文變化)
語序	丟失 (N-gram 可部分補救)	丟失	編碼順序與上下文 (受 context window 限制)
可解釋性	★ 高：每一維就是一個詞	中	低
算力需求	★ 極低	低	高
什麼時候用	小資料、要解釋、要稽核	詞層級的語意任務	語意與上下文重要時

★ 這張表把 M6 與 M7 串起來了。注意最後兩列：不是越新越好，是看你的限制條件在哪。

M6 → M7 → M8：同一條線的三個階段

M6 → M7 → M8：同一條線的三個階段



M6 (本週)

詞袋與 TF-IDF。可解釋、可稽核、不用 GPU。

M7 (下週)

Tokenizer 把字元 / 子詞映射成模型 ID，不等於「進階斷詞」；上下文嵌入則讓同一詞依語境改變。

M8 (第 13 週起)

RAG 延伸了文件切塊、檢索與相似度；實務還會加入混合檢索、重排序、來源引用與生成驗證。

★ 三週是同一條線

都在處理「文字怎麼變成電腦能算的東西」。
表示法從詞袋 → 向量 → 語意空間，一路變聰明。

Take-Home Assignment | M6 文字探勘

W12 上週前繳交，上週期成績 50%

作業說明

【任務一】前處理的影響實驗 (40%)

資料：AI Studio 的 amazon_alex.csv (或自選公開中文評論資料集)

1. 建立一個文本分類流程：讀檔 → 前處理 → TF-IDF → Naive Bayes
2. 做四組實驗，其他條件固定，只改前處理：
(a) 完全不做前處理 (b) 只斷詞 (c) 斷詞 + 去停用詞
(d) 斷詞 + 去停用詞 + 詞形還原
3. 記錄四組的準確率、Recall、F1，畫成一張表
4. 寫 200 字：哪一步帶來的提升最大？有沒有哪一步反而變差？

【任務二】中文斷詞與自訂詞典 (35%)

1. 自己寫 10 句含醫護專有名詞的句子 (虛構，不得用真實病歷)
2. 用 jieba 預設詞典斷詞，記錄哪些術語被切碎
3. 建立一份自訂詞典 (至少 15 個詞)，重新斷詞
4. 做成前後對照表，寫 150 字說明差異

【任務三】主題模型探索 (25%)

1. 找一份公開的中文評論或問卷開放題資料 (至少 300 筆)
2. 跑 NMF，主題數分別設 3、5、8
3. 為每個主題人工命名，說明你的命名依據
4. 寫 150 字：哪一個主題數最說得通？為什麼？

【繳交】PDF 一份，含流程截圖與結果表。檔名：M6_學號_姓名.pdf

【提醒】★ 一律使用公開資料集或虛構資料，不得使用任何真實病歷或個案紀錄。

本模組重點回顧

- 1 醫護領域最有價值的資料多半是文字，而你有看得懂它的優勢
- 2 NLP 三個時代並存：醫療今天仍大量使用規則式與 TF-IDF，因為可稽核
- 3 ★ 大量時間花在資料品質與前處理；詞級方法尤其依賴中文斷詞
- 4 自訂詞典常是詞袋流程的高投報率改進——但要用測試集驗證
- 5 TF-IDF 讓稀有詞浮出來：「病人」沒有鑑別力，權重最低
- 6 Naive Bayes 「天真」但快速；應與 Logistic / Linear SVM 一起當基準

查證資料與延伸閱讀 (Selected References)

方法與工具

[scikit-learn Text Classification](#) — 稀疏線性模型 / NB

[Hugging Face Tokenization](#) — word vs subword

[CKIP Transformers](#) — 繁中 WS / POS / NER

[BERTopic Documentation](#) — 模組化主題流程

[Hugging Face Pipelines](#) — zero-shot / 摘要 / NER

[GroupKFold](#) — 群組互斥切分

醫療、隱私與治理

[HHS HIPAA De-identification](#) — Safe Harbor 18 項

[臺灣個人資料保護法](#) — 直接 / 間接識別

[衛福部 2026 生成式 AI 指引](#) — 風險治理

[n2c2 2022 Clinical NLP](#) — patient-level split

[Wrenn et al., JAMIA 2022](#) — 臨床文字重複

[WHO 2024 AI for Health](#) — LMM governance

★ 「18 項 / 89 歲」屬美國 HIPAA Safe Harbor；臺灣專案應依個資法、IRB、院內政策與資料使用契約辦理。

M6 名詞速查表 NLP 與文字探勘

(1 / 4)

中文	English	一句話說明
自然語言處理	Natural Language Processing, NLP	讓電腦處理與理解人類語言的技術領域
文字探勘	Text Mining	從非結構化文字中萃取有用資訊與模式
語料庫	Corpus	用於分析或訓練的文本集合
斷詞 / 分詞	Word Segmentation	把連續文字切成可讀詞語；不同語料可能採不同詞界標準
自訂詞典	User Dictionary	補充斷詞器未收錄的專有名詞，醫療文本必備
停用詞	Stop Words	無實質意義的高頻詞；注意否定詞常誤列其中
詞形還原	Lemmatization	把詞還原為原形 (running → run) ；主要用於英文
子詞切分	Subword Tokenization	將文字切成介於字元與詞之間的 token，如 BPE、WordPiece、Unigram
詞性標註	Part-of-Speech Tagging, POS	標記名詞、動詞等詞性，可用於篩選

M6 名詞速查表 NLP 與文字探勘

(2 / 4)

中文	English	一句話說明
向量化	Vectorization	把文字轉成數值形式，供機器學習使用
詞袋模型	Bag of Words, BoW	只計次數、不管語序的文本表示法
文件—詞矩陣	Document-Term Matrix, DTM	列為文件、欄為詞的矩陣，值可為次數或權重
詞頻	Term Frequency, TF	某詞在該文件中出現的頻率
逆文件頻率	Inverse Document Frequency, IDF	出現在越少文件的詞權重越高
TF-IDF	Term Frequency–Inverse Document Frequency	詞頻乘以逆文件頻率，傳統文字探勘的標準權重
稀疏矩陣	Sparse Matrix	絕大多數元素為 0 的矩陣；DTM 的典型型態
N-gram	N-gram	連續 N 個詞的組合，可部分保留語序資訊
樸素貝氏	Naive Bayes	假設特徵條件獨立的分類器；文本分類上表現優異

M6 名詞速查表 NLP 與文字探勘

(3 / 4)

中文	English	一句話說明
拉普拉斯平滑	Laplace Smoothing	分子加 1、分母加 k，避免機率為零
主題模型	Topic Modeling	從無標籤文本中找出潛在主題
非負矩陣分解	Non-negative Matrix Factorization, NMF	把 DTM 分解為文件—主題與主題—詞兩矩陣
潛在狄利克雷分配	Latent Dirichlet Allocation, LDA	機率式主題模型，一文可屬多主題
情感分析	Sentiment Analysis	判斷文本的情緒傾向
面向情感分析	Aspect-Based Sentiment Analysis, ABSA	★ 針對特定面向分別判斷情緒，滿意度分析必備
命名實體辨識	Named Entity Recognition, NER	找出並標記人名、地名、日期、藥名等實體
否定偵測	Negation Detection	辨識「否認」「無」等否定語境；醫療文本關鍵
去識別化	De-identification	移除或代換可識別個人身分的資訊

M6 名詞速查表 NLP 與文字探勘

(4 / 4)

中文	English	一句話說明
零樣本分類	Zero-shot Classification	可對未見類別名稱做推論；不需任務訓練樣本，但仍需驗證
少樣本學習	Few-shot Learning	僅用少量範例即可完成任务
摘要生成	Summarization	把長文本濃縮成短摘要；屬生成式任務需人工覆核
文件相似度	Document Similarity	以向量距離衡量兩份文件的相近程度
餘弦相似度	Cosine Similarity	以向量夾角衡量相似度，文本分析常用
jieba	jieba	常用中文斷詞套件，輕量易擴充，簡體語料為主
CKIP Transformers	CKIP Transformers	中研院開發，以臺灣繁體語料訓練，含斷詞、詞性與 NER
spaCy	spaCy	主流的英文 NLP 工具庫，提供完整前處理流程
Hugging Face Pipeline	Hugging Face Pipeline	以數行程式碼呼叫預訓練模型完成常見 NLP 任務